

File I/O

Most content that is programmed is stored in memory, meaning once the user exits the program, the contents are disappeared. To get information actually accessible, we store them in files.

file = open("test.txt", "w")

opens/creates a file, assigns it to the variable "file" name of the file with the type of file (.txt) default value is "r" for "read", but we plan to write in the txt file.

open returns a file handle, which is a value that allows me to access the file subsequently.

file.write("something") ← the .write() method allows the user to write stuff into a file.

file.close() ← the .close() method closes and saves the file (Important)

The "w" parameter in the "open" function will not only create a file, but also recreate it, every time the file is opened in that "w" mode.

To keep adding contents to an existing file:

file = open("test.txt", "a")
file.write("something") ← "a" for "append"
file.close()

The .close() method is not always used to save files from corruption. Using with:

with open("test.txt", "a") as file:
... file.write("something")

"file.write('Something')" is now in the context of the "with" Statement, which ensures that once the code reaches the end of the indented blocks within the "with" Statement, the file automatically closes.

to read files using "with":

```
with open("test.txt", "r") as file:  
    ... lines = file.readlines()
```

"r" for "read" mode.

.readlines method reads the lines of a specified file and assigns them to a variable named "lines". "lines" holds a list of the lines as items.

to read a file simply loads a file, and does not save it.

It is possible to do multiple things within the "with" Statement.

ex. doing 2 things at different occasions:

```
with open("test.txt", "r") as file: } open the file and store each line  
    ... lines = file.readlines() } from "file" into "lines" as a list  
for line in lines: } for each item (line) in "lines", print the line.  
    ... print(line)
```

ex Cont. doing 2 things at once:

```
with open("test.txt", "r") as file: } open the file, and for every line  
    ... for line in file: } in "file", print the line.  
    ... print(line)
```

Both are dependent on the situation and the user. One is not better than the other.

ex. Sorting lines in a file, then printing

```
l = [] ← initialize a list to store items in memory  
with open("test.txt", "r") as file: } open and read the file, and  
    ... for line in file: } for each line in "file", append to  
    ... l.append(line.rstrip()) } the initialized list  
for i in sorted(l): } for each item in the sorted list,  
    ... print(i) } print the item
```


ex. cont. If we want to simply sort the file:
with open("test.txt") as file:
 for line in sorted(file):
 print(line.rstrip())

} Same thing as previous example,
but more compact...

CSV files (Comma Separated Values)

- a common convention to store multiple pieces of information in a format that uses commas to separate values (usually into blocks/cells)

ex. opening the CSV file using with:
with open("lefile.csv") as file:
 for line in file:
 leline = line.rstrip().split(",")

"leline" is a variable assigned to a list that holds items that have been separated by commas from a string.

Lists can be unpacked, even though a tuple is not present. However, the user must know beforehand how many values will be present in the list.

Python has a feature that allows one to pass functions as arguments into other functions.

ex. `sorted(leDict, key=unfunc)`

"key" is a parameter where its default value is the function: "unfunc"

By doing this, the user can keep some data together until it is actually used.

How "Sorted" function uses its "Key" parameter:

When a function is passed to the Sorted function as the value of "key", the function is automatically called by the Sorted function on each iteration of a list. It then uses the return value of the passed function to decide what string to actually use to compare to decide which is alphabetically correct.

ex. `lelist = [{'ch': 'a', 'num': 3},
 {'ch': 'b', 'num': 2},
 {'ch': 'c', 'num': 1}]`

* each dictionary is assigned to a global variable: "leDict"

```
def org1(leDict):  
    return leDict['ch']
```

```
def org2(leDict):  
    return leDict['num']
```

```
for it in sorted(lelist, key=org1):  
    print(f"{it['ch']} and {it['num']}")
```

```
>>> a and 3  
>>> b and 2  
>>> c and 1
```

Sorting based on the value of each `leDict['ch']`

```
for it in sorted(lelist, key=org2):  
    print(f"{it['ch']} and {it['num']}")
```

```
>>> c and 1  
>>> b and 2  
>>> a and 3
```

Sorting based on the value of each `leDict['num']`

The user can also use a lambda function instead of creating a function that will only be used once. A lambda function is an anonymous function that has no name.

ex. Syntax for org1: `lambda leDict: leDict['ch']`

ex cont. `lambda leDict: leDict['ch']`

`lambda` is a key word in Python that notifies the program of an anonymous function with no name. The anonymous function takes in a parameter.

`leDict` is the name of the parameter. It can be named anything I want. However, since I am using it in the context of the previous example, it is named "`leDict`". The name of the parameter will be used in the lambda function.

`leDict['ch']` is the return value. I am returning the value associated with the key "`leDict['ch']`".

I can also do this:

`lambda poop: poop['ch']`

this will work too,
but also assumes that
"`poop`" is a dictionary.

back to CSV...

If there is a comma (intentionally placed) that is not meant to be split up on: use Python's CSV file documentation

ex. line of interest: `Poobay, "kapoo, kapow"`

comma to split

comma of interest

Import the CSV module

`import csv`

`lelist = []`

`with open("leCSV.csv") as file:`

`.... reader = csv.reader(file)`

`.... for row in reader:`

`lelist.append({"name": row[0], "abi": row[1]})`

reader returns a row that is a list

Also make sure to quote the contents with the comma of interest.

Instead of using `CSV.Reader()`, the user can use `CSV.DictReader()`, which iterates over the given file top to bottom, where each line of text is not a list of columns, but a dictionary of columns. To use this, however, a header is necessary to exist in the CSV file.

ex. CSV file with headers:

lename, leabi ← header
poob, poop
plebiam, pee
fucker, no } Content

with open("le.csv") as file:
... read = CSV.DictReader(file)
... for row in read:
... lelist.append({'name': lename,
 'abi': leabi})

The user can also use the writer functions from the CSV file with `CSV.writer()`, which takes the file variable as its only argument. The function uses another line with the `.writerow([<contents>])` method, which writes the contents into the specified file. Important: use the append mode "a" to write to a file.

```
lewrite = CSV.writer(lefile)
lewrite.writerow([lename, leabi])
```

← contents inside a list,
which is inside a method

There is also `CSV.DictWriter()`, which uses the same `.writerow()` method, but with a dictionary inside. The function `CSV.DictWriter()` should take 2 parameters: the file variable and a list of header materials set as default to the parameter "fieldnames". Fieldnames is necessary because it provides the function information about the order of the columns of the file, so that when the orders change, it knows what to associate to.

```
lewrite = CSV.DictWriter(lefile, fieldnames=["lename", "leabi"])
lewrite.writerow({'leabi': <var1>, 'lename': <var2>})
```

order does not matter because dictionary keys
are defined in fieldnames parameter.

Binary files

- files full of 1s and 0s, which usually stores graphical, video, and audio information.

The python library: Pillow (PIL)

- navigates image files and performs operations on them

managing .gif: using multiple image files to give the illusion that the image is moving

ex. leplay - the gif ← access command-line arguments, aka the .gif files
import sys
from PIL import Image treat the files as images using "Image"

```
images = []  
for arg in sys.argv[1:]:  
    ... image = Image.open(arg)  
    ... images.append(image)
```

} using a for loop, iterate over arguments in each command-line argument, open the argument with the "Image" function, and append to the list

```
images[0].save(  
    ... "leplays.gif", save_all=True, append_images=[images[1:]],  
    duration=200, loop=0  
)
```

Save the first image, but also append another image (images[1:]), Save all of the files together. Pause 200 milliseconds in between each frame, and loop infinitely.